Beat Claude: General 000

Candidate: Wes Jones | **Target role:** AI Automation Engineer | **Brief version:** 2026-07

1. Written Answer

The role I want

I want to own the path from an operational problem to a working AI-assisted system: observe the current process, identify the actual constraint, size the opportunity, build the workflow, test ugly cases, deploy it with named owners and controls, and stay close enough to learn where users route around it. My best work sits between operations and engineering. I am not looking to write specifications for someone else to implement; I want to build the thing and be accountable for whether it survives real use.

Single Grain is interesting because the stated transition from agency to tech-enabled services creates a concrete operating problem. Repetitive service work should become governed execution, while judgment, client context, and consequential approvals remain human. That is the same boundary I have been working on: intake, routing, execution, review, escalation, and evidence across systems that were not designed as one product.

My first 90 days

[Assumed] In days 1-30, I would map a small set of recurring workflows with the people doing them, record baseline frequency and failure modes, and create a portfolio view that separates deletion, process repair, assistance, and full automation. I would also document approved models, data boundaries, access patterns, logging expectations, and a common production-readiness gate.

[Assumed] In days 31-60, I would ship one narrow workflow with a clear owner, measurable baseline, reversible actions, exception routing, and a human approval point where consequence or ambiguity is high. I would favor a process that is frequent enough to learn from but bounded enough that failure does not damage a client relationship.

[Assumed] In days 61-90, I would stabilize the first system, compare expected and actual value, retire weak assumptions, and use what we learned to ship a second workflow faster. The output would be more than two automations: it would be a reusable intake, build, validation, release, and maintenance pattern.

Proof of ability

[Observed] I built a compliance-oriented workflow covering 42 control checks with a 99 percent automated pass rate, Slack approval loops, and Monday.com/Notion evidence trails. This is a candidate-stated production result; the underlying records are private and therefore listed as Tier 0 in the evidence log rather than represented as public proof.

[Observed] My customer-facing automation work has covered 50-plus API/OAuth providers and 30-plus connected systems. The transferable lesson was not the number of tools. It was learning to isolate trigger, mapping, authentication, payload, rate-limit, and ownership failures before replacing the platform. These are candidate-stated experience counts and are Tier 0 here because customer records cannot be published.

I also built a reusable regression workflow for customer-facing teams to reproduce recurring failures, preserve evidence, and distinguish configuration problems from likely product defects before engineering escalation. The public artifact in this repository is a safe, synthetic demonstration of the same operating judgment: incomplete requests are blocked, consequential actions require review, and every decision has an inspectable reason. Review `src/workflow_control_harness.py`, `data/automation_requests.json`, and `output/portfolio_report.json` to inspect and reproduce the public demonstration.

My AI edge

My daily stack includes Claude Code, OpenAI Codex, Python, JavaScript, APIs, Zapier, and business systems such as HubSpot, Google Workspace, Salesforce, Slack, Airtable, Notion, and Monday.com. I use coding agents as implementation partners: first to explore a bounded problem, then to produce code or configuration, then to challenge the result with tests and failure cases. I keep the business rule, acceptance criteria, release decision, and consequential approval human-owned.

My best AI win is the requirements-to-workflow builder behind my recent work. It turns open-ended requirements into connected actions with validation, human approval, traceability, and reusable implementation patterns. AI made it practical to move from ambiguous operating language to a tested implementation quickly; the hard part remained deciding what the system was allowed to do, how a failure became visible, and who owned the exception.

[Estimated] In the next 2 years, low-complexity workflow assembly will become table stakes. The scarce role will shift from "person who can connect tools" to "person who can model the operation, set boundaries, build the control plane, and own the result." Pure handoff work will shrink. More teams will need builders who can combine process discovery, integration depth, evaluation, access control, observability, and adoption.

2. Operating Artifact

The operating artifact is the runnable Workflow Control Harness in this repository. It accepts synthetic automation requests and run events, validates each request, calculates a transparent priority score, applies a production-readiness gate, routes exceptions, and writes an auditable report.

The non-obvious choice is that priority and deployability are separate. A valuable request can rank highly and still be blocked because it lacks a named owner, evidence sink, retry policy, alert route, validation rule, or required approval. That prevents a polished prototype from being mistaken for an operable system.

[Observed] The included synthetic fixture contains 5 requests. Running the command in `README.md` produces `output/portfolio_report.json` and `output/portfolio_report.md`; the unit-test command exercises scoring, readiness blocking, and exception detection. These counts are directly inspectable in the repository and constitute a Tier 2 demo artifact plus Tier 3 source records and test logs when run.

3. Evidence Log

Claim	Source label	Evidence tier	Reviewer access
Compliance workflow covered 42 checks and reached a 99 percent automated pass rate	[Observed] candidate production record	Tier 0 in this public packet	Private records are not published; discuss and verify only through an authorized walkthrough
Automation experience spans 50-plus API/OAuth providers and 30-plus connected systems	[Observed] candidate work history	Tier 0 in this public packet	Customer records are confidential; employment history and examples can be discussed live
The harness separates prioritization from production readiness	[Observed] repository behavior	Tier 2	`src/workflow_control_harness.py` and `output/portfolio_report.md`
The harness detects seeded missing controls and failed run events	[Observed] repository test run	Tier 3	`data/`, `tests/`, generated outputs, and commit history
The fixture contains 5 synthetic requests	[Observed] repository fixture	Tier 3	`data/automation_requests.json`

4. Number Source Labels

- **[Observed]** means measured in prior work or directly reproduced from this repository. Private prior-work observations are explicitly limited to Tier 0 here when the source cannot be shared. Public observed inputs and outputs are inspectable in `data/automation_requests.json` and `output/portfolio_report.json`.
- **[Estimated]** means my reasoned forecast.
- **[Benchmarked]** would mean a named outside source; no benchmarked numeric claim is used in this packet.
- **[Assumed]** means a planning constraint chosen to make the operating proposal concrete.

5. AI Usage Disclosure

I used OpenAI Codex to help inspect the public challenge requirements, structure the repository, draft portions of the code and prose, and run validation. I chose the artifact, operating boundaries, evidence treatment, scoring factors, seeded failure cases, and which claims were supportable. I checked the generated code by running the harness and its tests, reviewed the output for internal consistency, and removed any claim that would require invented evidence. The main weak spot is that the public artifact uses synthetic data; it proves how I build and reason, not the private production metrics from earlier work.

6. Failure Handling / What Breaks

The score is only as honest as the inputs. A requester can exaggerate hours saved, hide exception frequency, or understate data sensitivity. The harness responds by keeping assumptions visible and by refusing to treat score as deployability. Before release, I would verify the baseline with source records, sample real edge cases, confirm access scope, and assign an owner who can stop the workflow.

Upstream schema changes, expired credentials, rate limits, model drift, prompt injection in untrusted content, and silent partial writes can all make a previously healthy automation unsafe. Detection should combine validation at boundaries, idempotency where possible, retry limits, dead-letter routing, run logs, alerts, and periodic review. A failed or ambiguous consequential action should stop and route to a human rather than guess.

7. Artifact Access

The repository is public and requires no login. A reviewer can reproduce the output with the two commands in `README.md`. All data is synthetic, setup uses the Python standard library, and no API key is required. The generated report is committed for a no-setup review path, while the source, fixtures, and tests make the result reproducible.

What stays human

Humans retain approval for external publication, client-impacting changes, access grants, destructive actions, financial commitments, and ambiguous exceptions. Humans also decide whether the process should exist at all. AI may propose, transform, classify, and summarize; it should not convert weak context into irreversible authority.